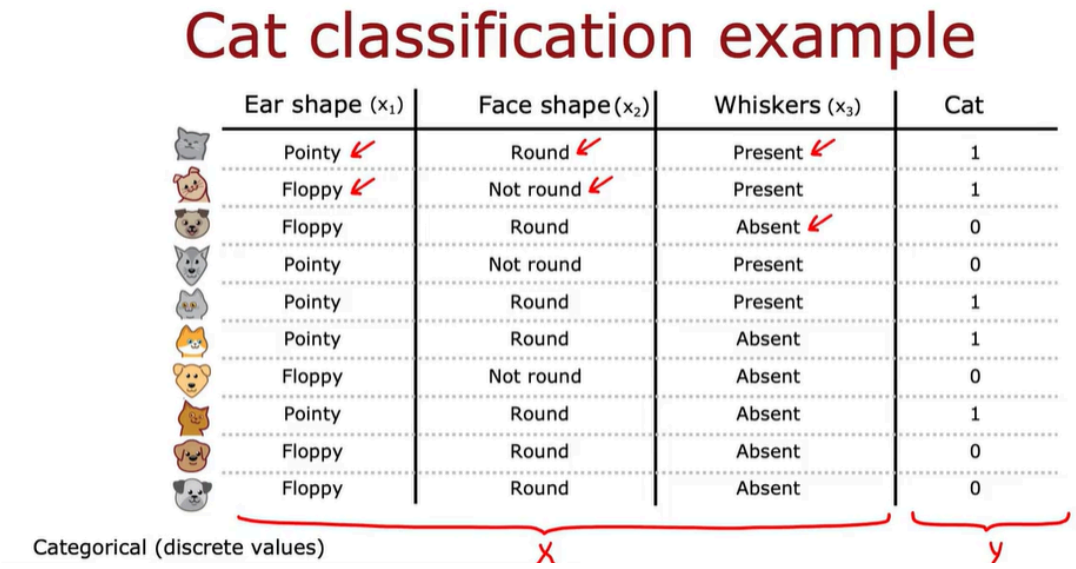
Decision Trees and Decision Tree Learning

Decision tree model

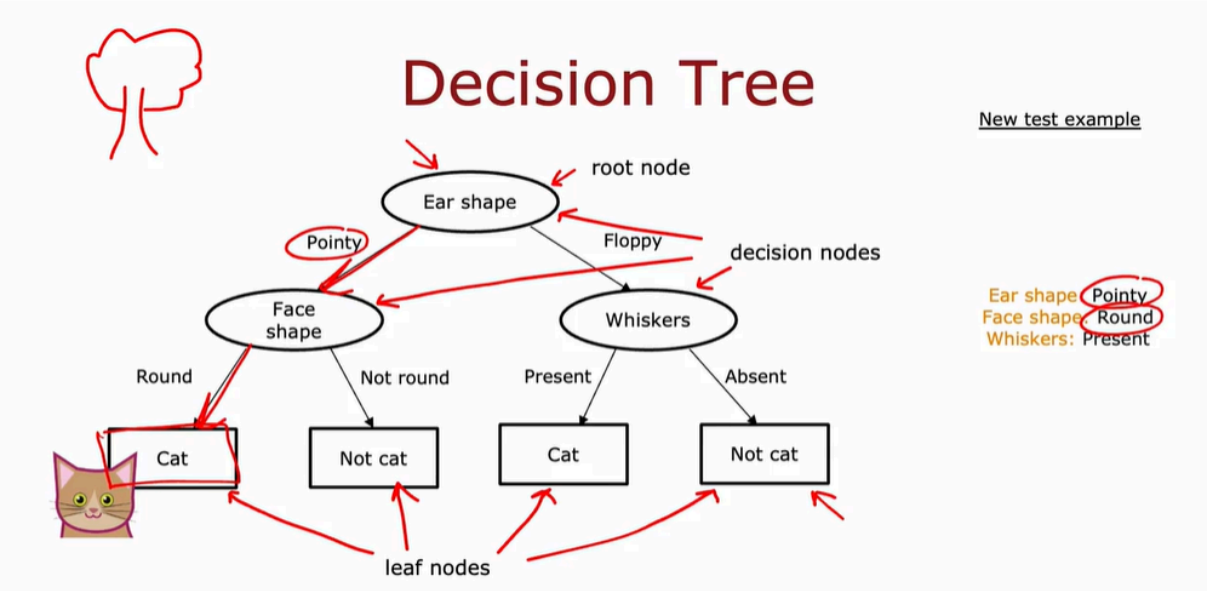


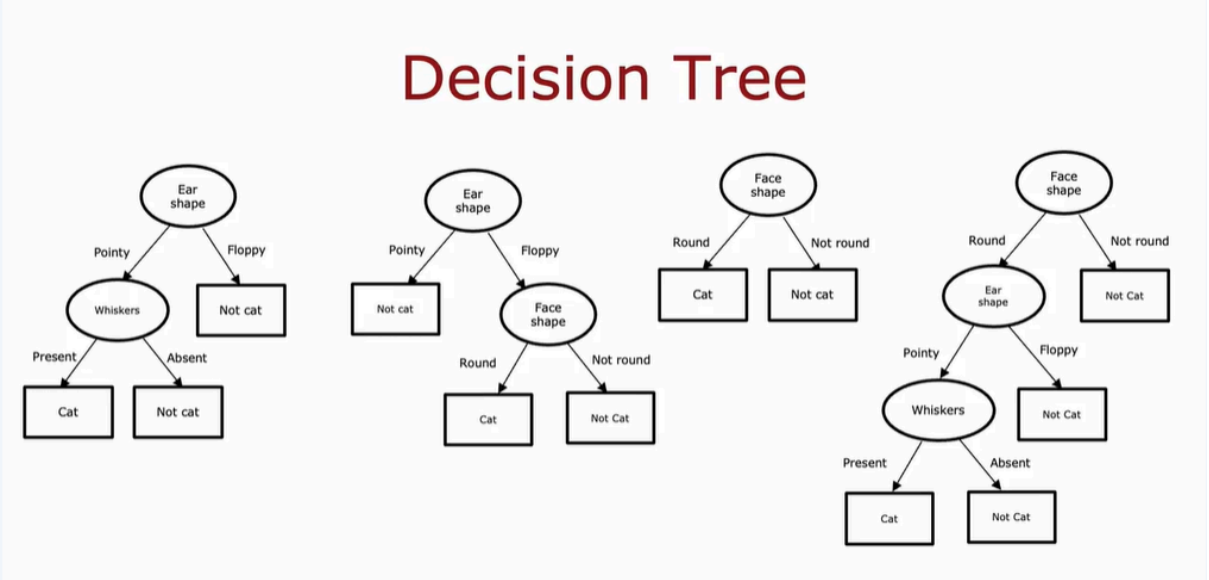
I'm going to use as a running example this week a cat classification example. You are running a cat adoption center and given a few features, you want to train a classifier to quickly tell you if an animal is a cat or not.

- I have here 10 training examples. Associated with each of these 10 examples, we're going to have features regarding the animal's ear shape, face shape, whether it has whiskers, and then the ground truth label that you want to predict this animal cat. The first example has pointy ears, round face, whiskers are present, and it is a cat.
- The second example has floppy ears, the face shape is not round, whiskers are present, and yes, that is a cat, and so on for the rest of the examples.
- This dataset has five cats and five dogs in it. The input features X are these three columns, and the target output that you want to predict, Y, is this final column of, is this a cat or not? In this example, the features X take on categorical values. In other words, the features take on just a few discrete values. Your shapes are either pointy or floppy. The face shape is either

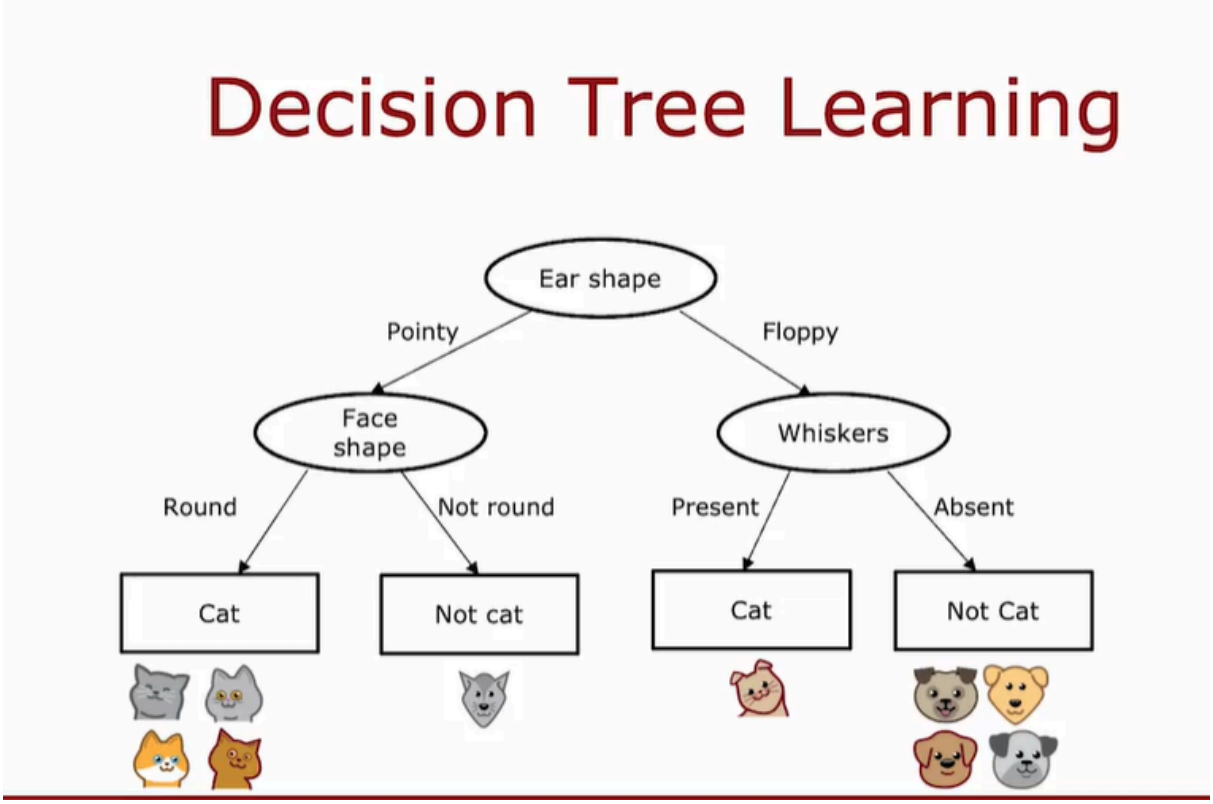
round or not round and whiskers are either present or absent. This is a binary classification task because the labels are also one or zero.

Decision Tree example:

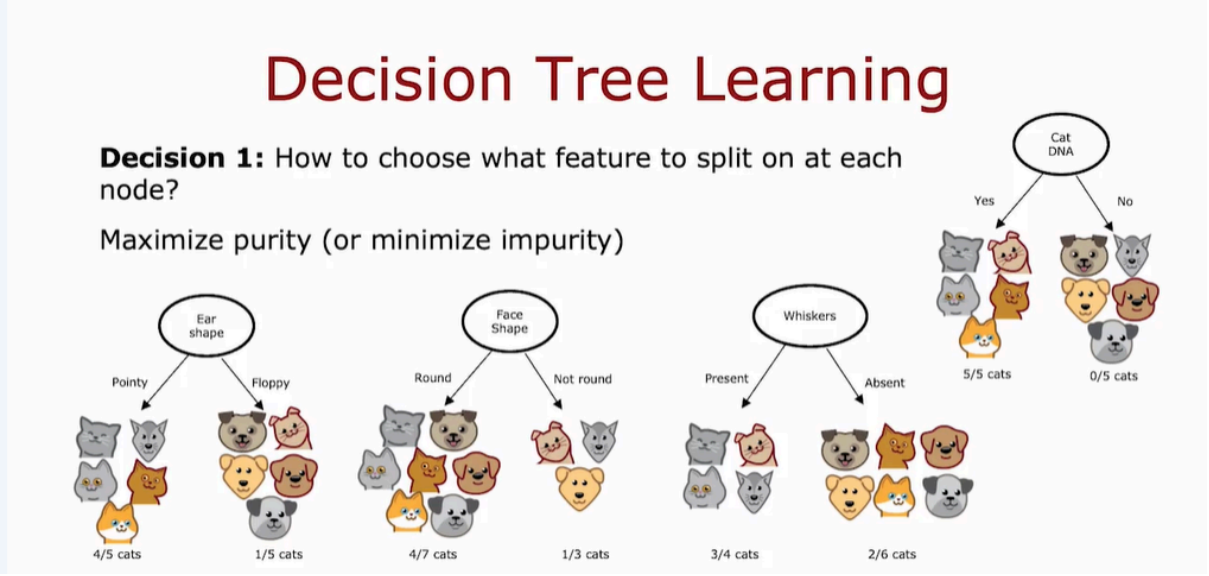




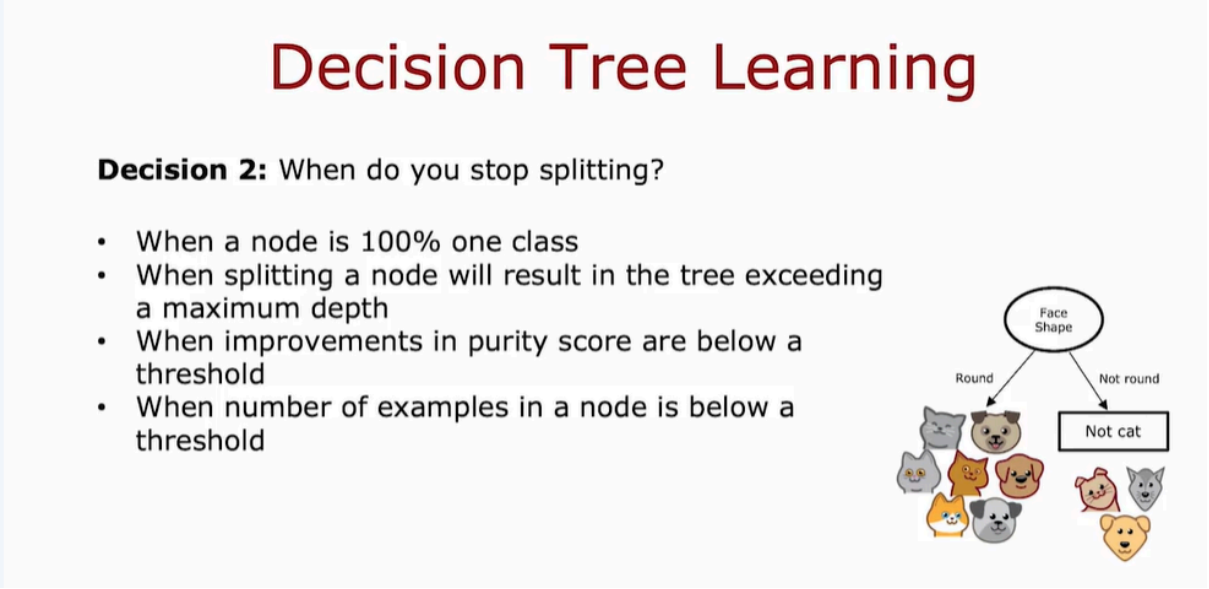
Learning Process



Decision 1: how to choose what feature to split on at each node?



Decision 2: When do you stop spliting

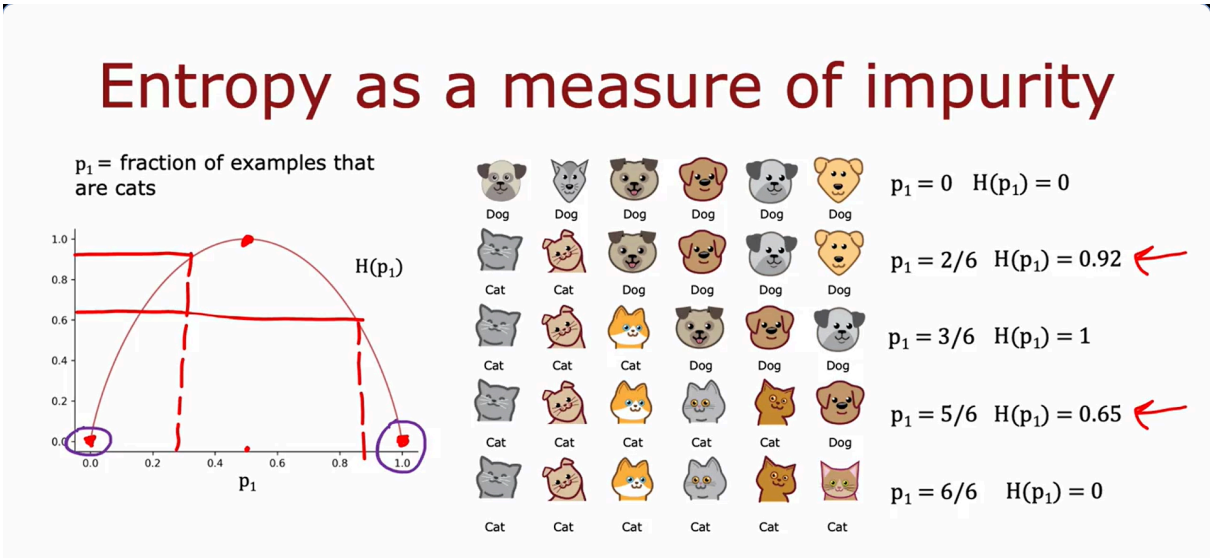


For example, if at the root node we have split on the face shape feature, then the right branch will have had just three training examples with one cat and two dogs and rather than splitting this into even smaller subsets, if you decided not to split further set of examples with just three of your examples, then you will just create a decision node and because there are mainly dogs, 2/3 number of dogs here, this would be a node and this makes a prediction of not cat. Again, one reason you might decide this is not worth splitting on is to keep the tree smaller and to avoid overfitting.

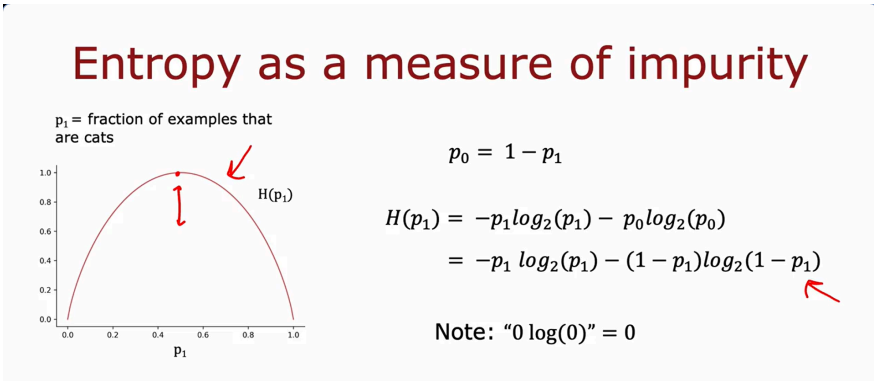
Measuring purity

"Purity" là một
chỉ số đánh giá chất lượng phân cụm (clustering), đặc biệt dùng trong các bài toán **phân cụm không giám sát** (unsupervised learning). Nó giúp đo mức độ mà các cụm (clusters) được tạo ra **chỉ chứa các phần tử từ cùng một lớp thực sự** (true class).

Entropy as a measure of impurity



Let's denote p_0 = the fraction of examples that are not cats. $p_0 = 1 - p_1$



If you plot the function:

$$H(p_1) = -p_1 \log_2(p_1) - (1 - p_1) log_2(1 - p_1)$$

in a computer, you will find that it will be exactly this function on the left.

if p_1 or p_0 = 0 then an expression like this will look like $0 \log_2(0)$, and $\log_2(0)$ is technically undefined, it's actually negative infinity. But by convention for the purposes of computing entropy, we'll take $0 \log_2(0) = 0$ and that will correctly compute the entropy as zero or as one to be equal to zero.

If you're thinking that this definition of entropy looks a little bit like the definition of the logistic loss that we learned about in the last course, there is actually a mathematical rationale for why these two formulas look so similar. But you don't have to worry about it and we won't get into it in this class.

Summary

Entropy function

- is a measure of the impurity of a set of data. It starts from zero, goes up to one, then comeback down to zero as a function of the fraction of positive examples in your sample.

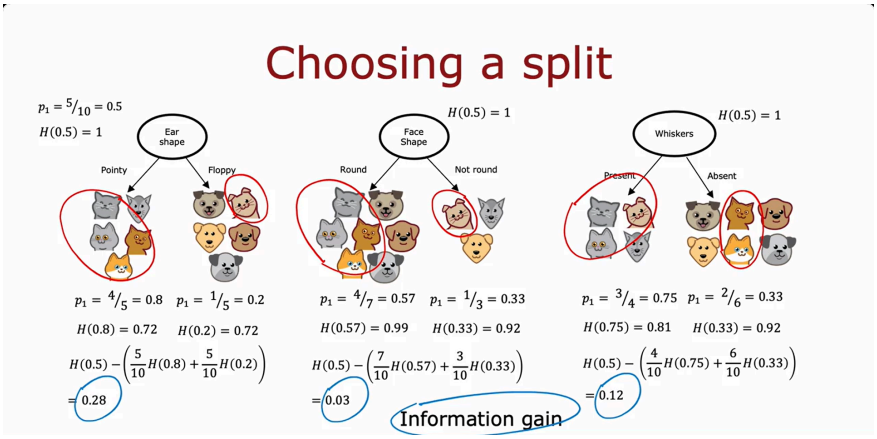
For example, if you look in open source packages you may also hear about something called the Gini criteria, which is another function that looks a lot like the entropy function, and that will work well as well for building decision trees.

Choosing a split: Information Gain

When building a decision tree, the way we'll decide what feature to split on at a node will be based on what choice of feature reduces entropy the most.

Reduces entropy or reduces impurity, or maximizes purity. In decision tree learning, the reduction of entropy is called **information gain**.

Information gain



Let's use the example of deciding what feature to use at the root node of the decision tree we were building just now for recognizing cats versus not cats.

- If we had split using their **ear shape** feature at the root node, this is what we would have gotten, five examples on the left and five on the right. On the left, we would have 4/5 cats, so P1 = 4/5 = 0.8. On the right, one out of five are cats, so P1 = 1/5 = 0.2. If you apply the entropy formula from the last video to this left subset of data and this right subset of data, we find that the degree of impurity on the left is entropy of 0.8, which is about 0.72,

and on the right, the entropy of 0.2 turns out also to be 0.72. This would be the entropy at the left and right subbranches if we were to split on the ear shape feature.

- One other option would be to split on the **face shape** feature. If we'd done so then on the left, 4/7 examples would be cats, so P1 is 4/7. And on the right, 1/3 are cats, so P1 on the right is 1/3. The entropy of 4/7 and the entropy of 1/3 are 0.99 and 0.92. So the degree of impurity in the left and right nodes seems much higher, 0.99 and 0.92 compared to 0.72 and 0.72.
- Finally, the third possible choice of feature to use at the root node would be the whiskers feature in which case you split based on whether whiskers are present or absent. In this case, P1 on the left is 3/4, P1 on the right is 2/6, and the entropy values are as follows.

The key question we need to answer is, given these three options of a feature to use at the root node, which one do we think works best? It turns out that rather than looking at these entropy numbers and comparing them, it would be useful to take a weighted average of them, and here's what I mean. If there's a node with a lot of examples in it with high entropy that seems worse than if there was a node with just a few examples in it with high entropy. Because entropy, as a measure of impurity, is worse if you have a very large and impure dataset compared to just a few examples and a branch of the tree that is very impure.

The key decision is, of these three possible choices of features to use at the root node, which one do we want to use? Associated with each of these splits is two numbers, the entropy on the left sub-branch and the entropy on the right sub-branch. In order to pick from these, we like to actually combine these two numbers into a single number. So you can just pick of these three choices, which one does best? The way we're going to combine these two numbers is by taking a weighted average. Because how important it is to have low entropy in, say, the left or right sub-branch also depends on how many examples went into the left or right sub-branch. Because if there are lots of examples in, say, the left sub-branch then it seems more important to make sure that that left sub-branch's entropy value is low.

In this example(the example in the left) we have, 5/10 examples went to the left sub-branch, so we can compute the weighted average as $\frac{5}{10}H(0.8)$, and then add to that 5/10 examples also went to the right sub-branch, plus $\frac{5}{10}H(0.2)$.

Now, for this example in the middle, the left sub-branch had received seven out of 10 examples. and so we're going to compute $\frac{7}{10}H(0.57)$ plus, the right sub-branch had three out of 10 examples, so plus $\frac{3}{10}H(0.33)$.

Finally, on the right, we'll compute $\frac{4}{10}H(0.75)$ + $\frac{6}{10}H(0.33)$. The way we will choose a split is by computing these three numbers and picking whichever one is lowest because that gives us the left and right sub-branches with the lowest average weighted entropy.

In the way that decision trees are built, we're actually going to make one more change to these formulas to stick to the convention in decision tree building, but it won't actually change the outcome. Which is rather than computing this weighted average entropy, we're going to compute the reduction in entropy compared to if we hadn't split at all. If we go to the root node, remember that the root node we have started off with all 10 examples in the root node with five cats and dogs, and so at the root node, we had p_1 = 5/10 = 0.5. The entropy of the root nodes, $H(0.5) = 1$. This was maximum impurity because it was five cats and five dogs.

The formula that we're actually going to use for choosing a split is not this weighted entropy at the left and right sub-branches, instead is going to be the entropy at the root node, which is entropy of 0.5, then minus this formula ($\frac{5}{10}H(0.8)$ + $\frac{5}{10}H(0.2)$) → $H(0.5) - (\frac{5}{10}H(0.8)$ + $\frac{5}{10}H(0.2)) = 0.28$. In this example, if you work out the math, it turns out to be 0.28.

For the face shape example, we can compute entropy of the root node, H(0.5) minus this, which turns out to be 0.03,

and for whiskers, compute that, which turns out to be 0.12.

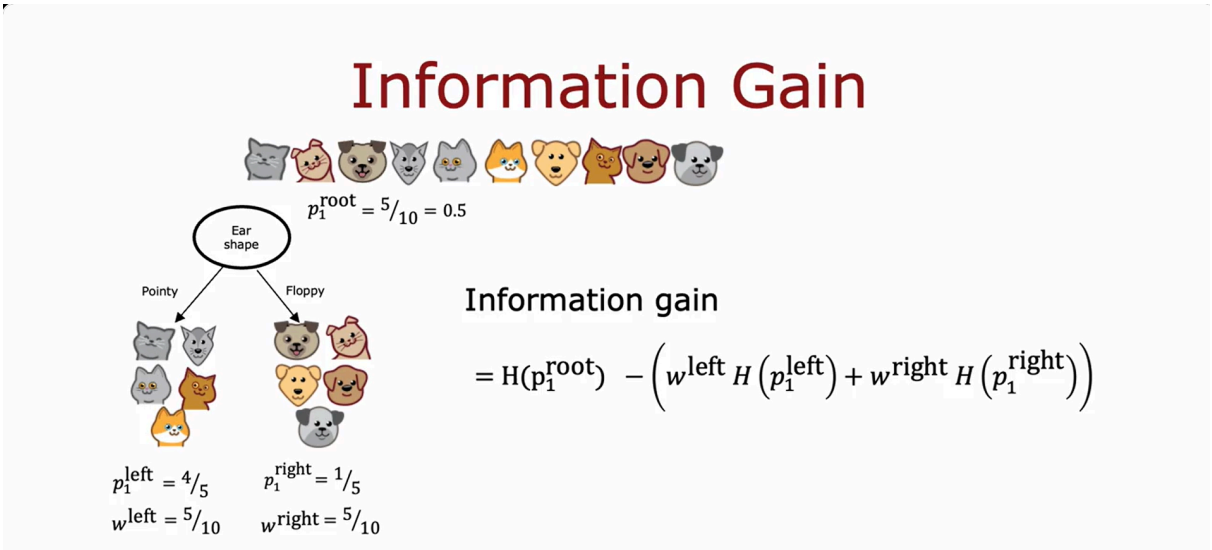
These numbers that we just calculated, 0.28, 0.03, and 0.12, these are called the information gain, and what it measures is the reduction in entropy that you get in your tree resulting from making a split. Because the entropy was originally one at the root node and by making the split, you end up with a lower value of entropy and the difference between those two values is a reduction in entropy, and that's 0.28 in the case of splitting on the ear shape.

Why do we bother to compute reduction in entropy rather than just entropy at the left and right sub-branches? It turns out that one of the stopping criteria for deciding when to not bother to split any further is if the reduction in entropy is too small.

In which case you could decide, you're just increasing the size of the tree unnecessarily and risking overfitting by splitting and just decide to not bother if the reduction in entropy is too small or below a threshold. In this other example, spitting on ear shape results in the biggest reduction in entropy, 0.28 is bigger than 0.03 or 0.12 and so we would choose to split onto ear shape feature at the root node.

Formula of information gain:

- Information gain measures the reduction in entropy that you get in your tree resulting from making a split. it makes the Decision Tree more purity.



Putting it together

Decision Tree Learning

Decision Tree Learning

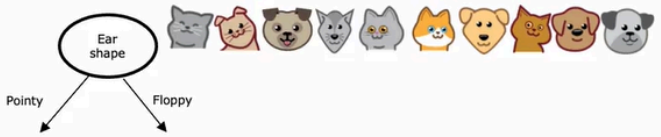
- Start with all examples at the root node
- Calculate information gain for all possible features, and pick the one with the highest information gain
- Split dataset according to selected feature, and create left and right branches of the tree
- Keep repeating splitting process until stopping criteria is met:
  - When a node is 100% one class
  - When splitting a node will result in the tree exceeding a maximum depth
  - Information gain from additional splits is less than threshold
  - When number of examples in a node is below a threshold

Recursive splitting

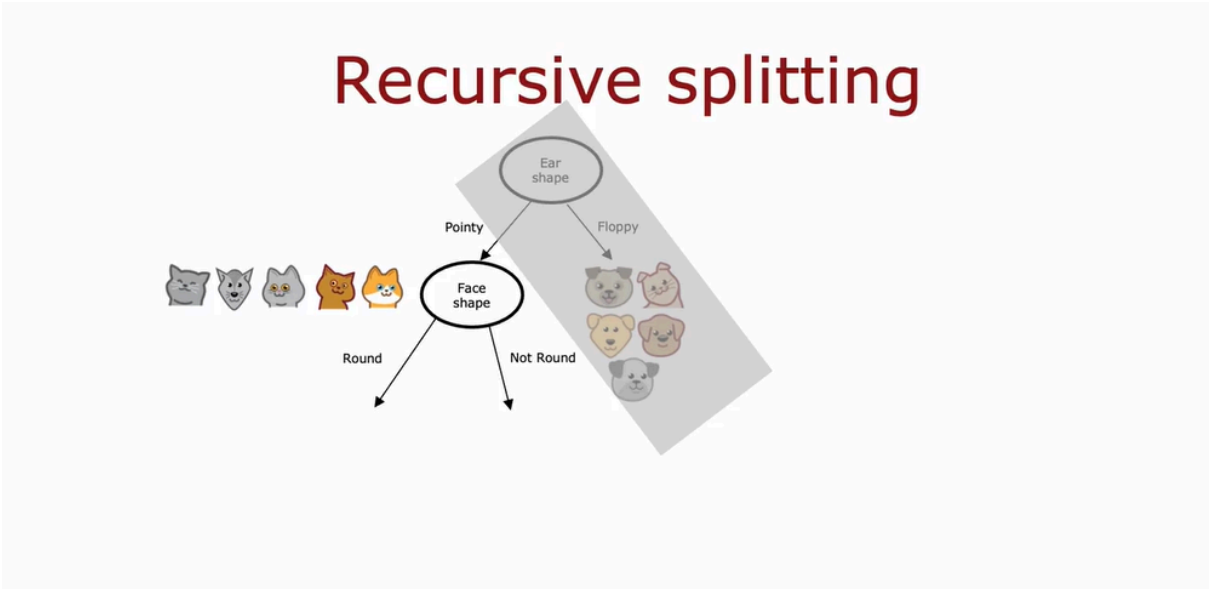
if we meet the stoping criteria, we put the leaf node at that criteria, follow the rule left → node → right

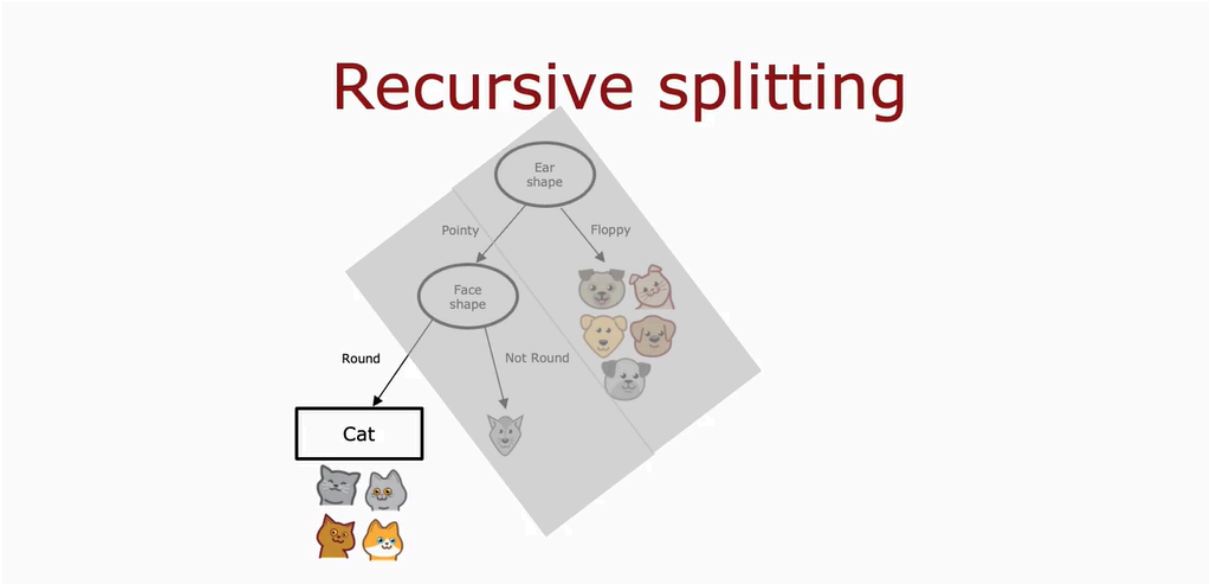
Recursive splitting

Recursive splitting

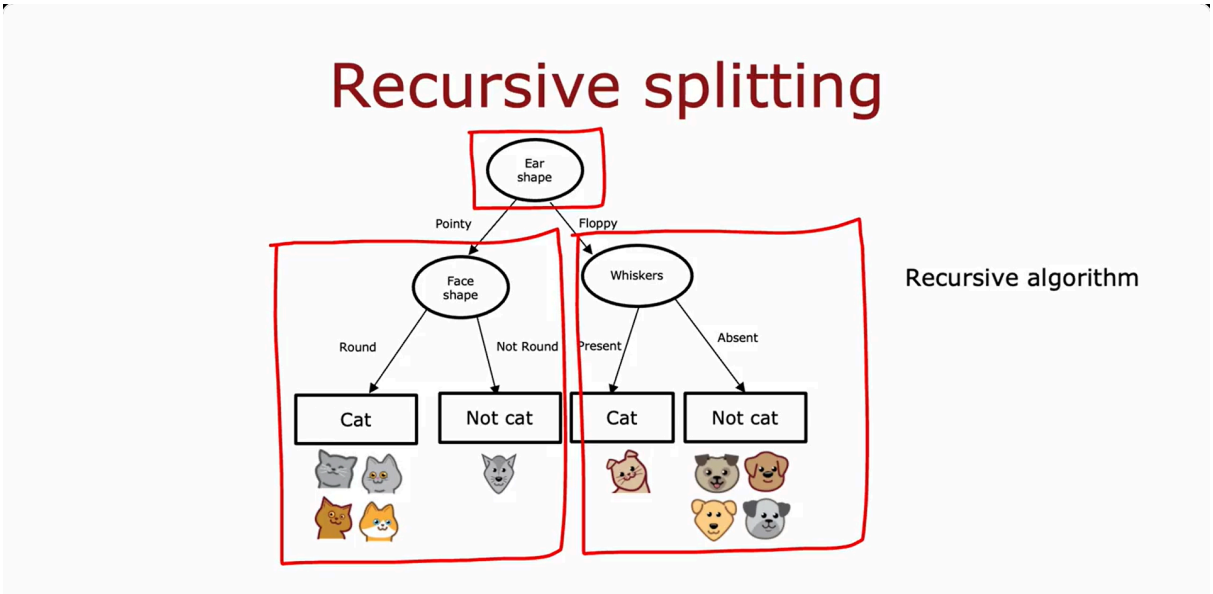


Recursive splitting





... after a long time



The way we built the right subtree was by, again, building a decision tree on a subset of five examples. In computer science, this is an example of a recursive algorithm. All that means is the way you build a decision tree at the root is by building other smaller decision trees in the left and the right sub-branches. Recursion in computer science refers to writing code that calls itself.

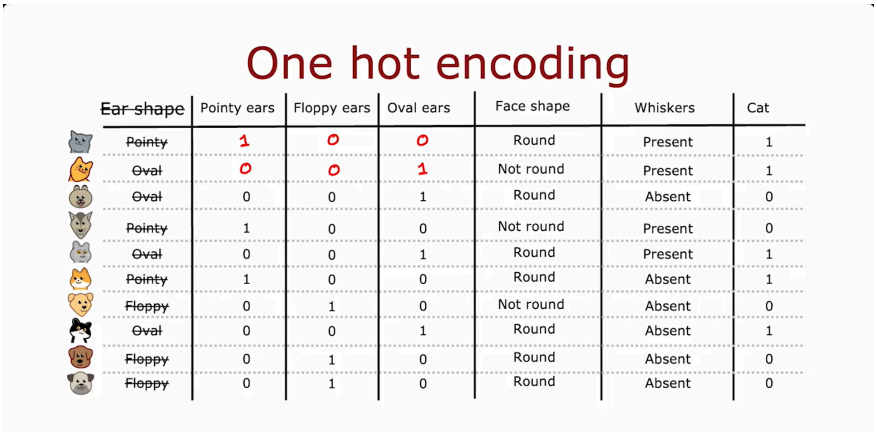
The way this comes up in building a decision tree is you build the overall decision tree by building smaller sub-decision trees and then putting them all together. That's why if you look at software implementations of decision trees, you'll see sometimes references to a recursive algorithm. But if you don't feel like you've fully understood this concept of recursive algorithms, don't worry about it. You still be able to fully complete this week's assignments, as well as use libraries to get decision trees to work for yourself.

But if you're implementing a decision tree algorithm from scratch, then a recursive algorithm turns out to be one of the steps you'd have to implement. By the way, you may be wondering how to choose the maximum depth parameter.

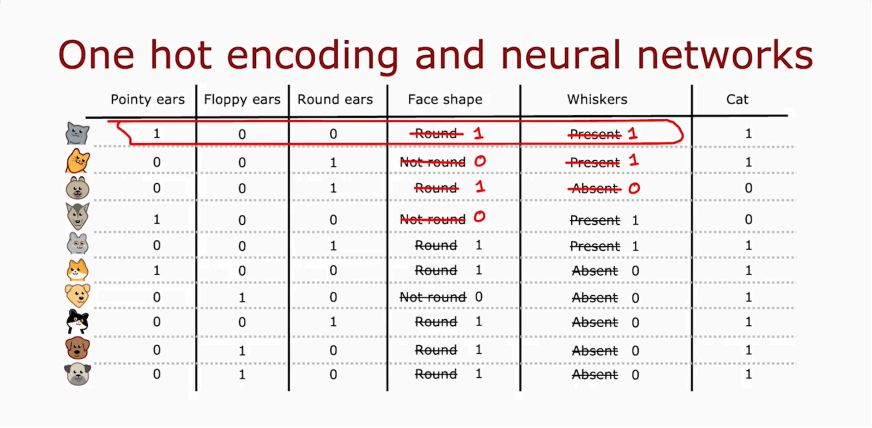
There are many different possible choices, but some of the open-source libraries will have good default choices that you can use. One intuition is, the larger the maximum depth, the bigger the decision tree you're willing to build. This is a bit like fitting a higher degree polynomial or training a larger neural network. It lets the decision tree learn a more complex model, but it also increases the risk of overfitting if this fitting a very complex function to your data.

In theory, you could use cross-validation to pick parameters like the maximum depth, where you try out different values of the maximum depth and pick what works best on the cross-validation set. Although in practice, the open-source libraries have even somewhat better ways to choose this parameter for you. Or another criteria that you can use to decide when to stop splitting is if the information gained from an additional split is less than a certain threshold. If any feature is splited on, achieves only a small reduction in entropy or a very small information gain, then you might also decide to not bother. Finally, you can also decide to stop splitting when the number of examples in the node is below a certain threshold. That's the process of building a decision tree.

Using one-hot encoding of categorical features



> If a categorial feature can take on k values, create k binary features (0 or 1 valued).



And you notice that among all of these three features, if you look at any role here (pointy ears or floppy ears or round ears), exactly 1 of the values is equal to 1.

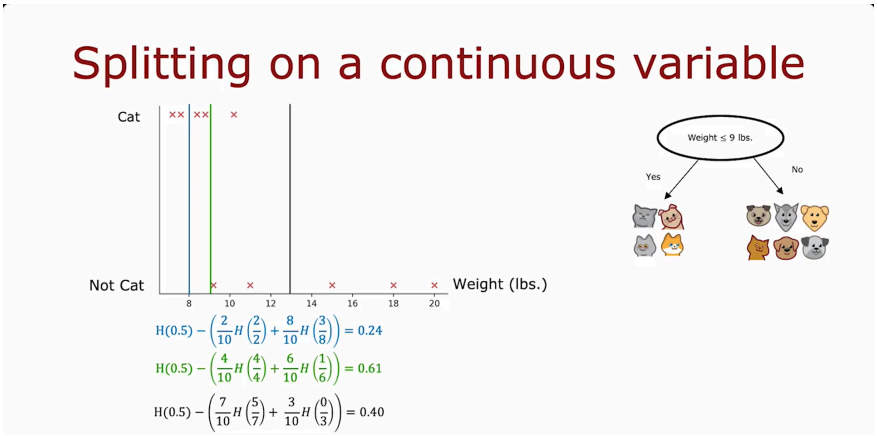
And that's what gives this method of feature construction the name one-hot encoding. And because one of these features will always take on the value 1 that's the hot feature and hence the name one-hot encoding. And with this choice of features we're now back to the original setting of where each feature only takes on one of two possible values, and so the decision tree learning algorithm that we've seen previously will apply to this data with no further modifications.

Just an aside, even though this week's material has been focused on training decision tree models the idea of using one-hot encodings to encode categorical features also works for training neural networks. In particular if you were to take the face shape feature and replace round and not round with 1 and 0 where round gets matter 1, not round gets matter 0 and so on. And for whiskers similarly replace presence with 1 and absence with 0.

They noticed that we have taken all the categorical features we had where we had three possible values for ear shape, two for face shape and one for whiskers and encoded as a list of these five features. Three from the one-hot encoding of ear shape, one from face shape and from whiskers and now this list of five features can also be fed to a new network or to logistic regression to try to train a cat classifier.

So one-hot encoding is a technique that works not just for decision tree learning but also lets you encode categorical features using ones and zeros, so that it can be fed as inputs to a neural network as well which expects numbers as inputs. So that's it, with a one-hot encoding you can get your decision tree to work on features that can take on more than two discrete values and you can also apply this to neural networks or linear regression or logistic regression training.

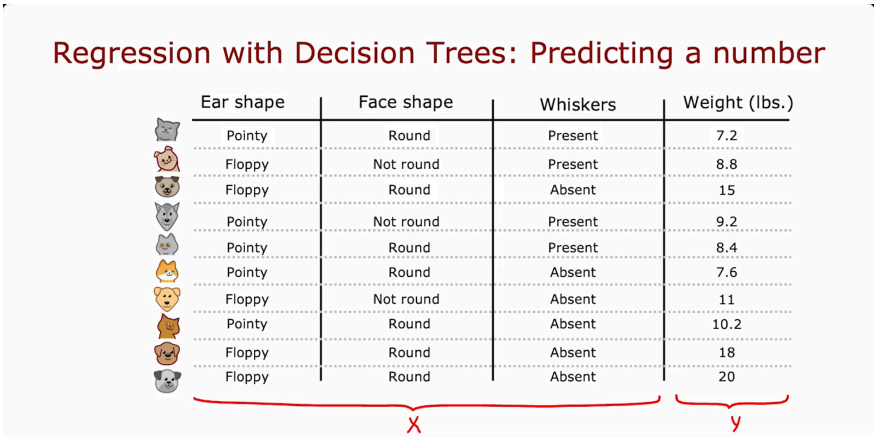
Continuous valued features



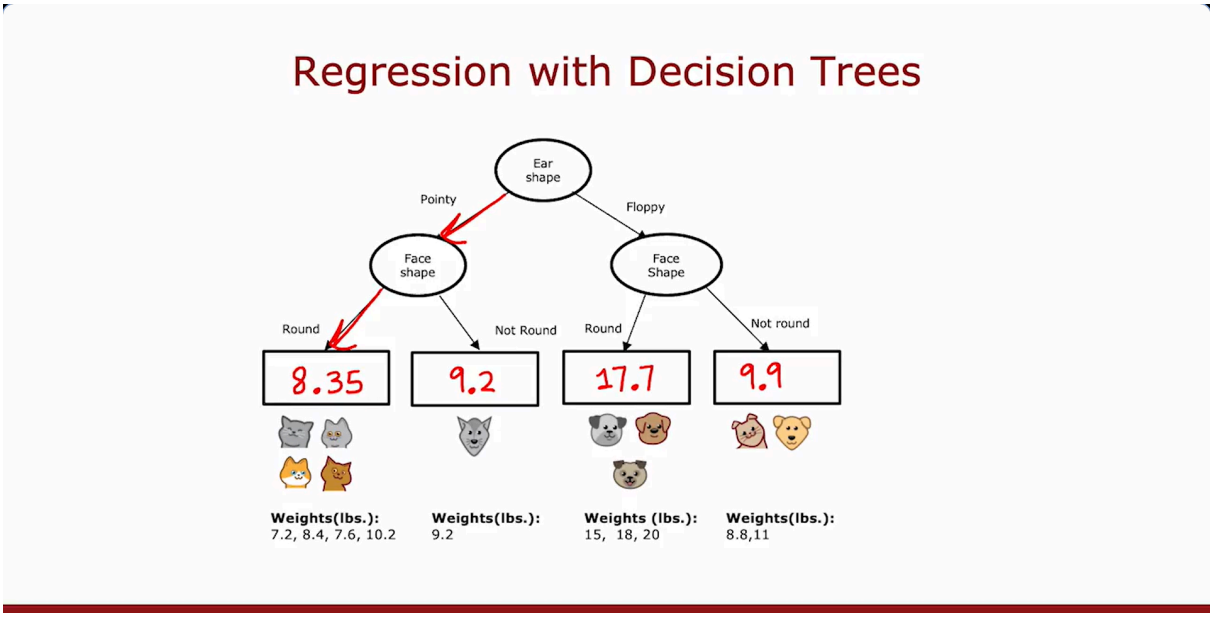
Summarize to get the decision tree to work on continuous value features at every node.

When consuming splits, you would just consider different values to split on, carry out the usual information gain calculation and decide to split on that continuous value feature if it gives the highest possible information gain. So that's how you get the decision tree to work with continuous value features. Try different thresholds, do the usual information gain calculation and split on the continuous value feature with the selected threshold if it gives you the best possible information gain out of all possible features to split on.

Regression Trees



How to find the y?



The last thing we need to fill in for this decision tree is if there's a test example that comes down to this node, what is there weights that we should predict for an animal with pointy ears and a round face shape? The decision tree is going to make a prediction based on taking the average of the weights in the training examples down here. And by averaging these four numbers, it turns out you get 8.35.